

Chapter 4

Representing Data & Engineering Features

Turning raw data into features
models can use

Learning Objectives

- Encode **categorical** variables correctly (and avoid two classic bugs)
- Use **binning**, **interactions**, and **polynomials** to enrich linear models
- Apply **nonlinear transformations** to skewed features
- **Select** the most useful features automatically
- Fold in **expert knowledge** as new features

How you represent your data often matters more than which model you pick.

Continuous vs. Categorical Features

Continuous

- Quantities that vary smoothly
- Pixel brightness, petal length, hours worked

Categorical (discrete)

- A value from a **fixed set**, no natural order
- Product brand, color, department

The distinction mirrors regression vs. classification — but on the **input** side.

Most raw data is **not** a clean array of floats.

Feature Engineering

Feature engineering = representing your data in the best way for the task.

- One of the main jobs of a data scientist
- The right representation can beat a better algorithm
- Two feature types → two toolkits, covered in this chapter:
 - encode **categoricals**
 - transform & select **features**

Part 1

Categorical Variables

The **adult** Income Dataset

Predict whether a worker earns **>50K** or **≤50K** (1994 US census).

Feature	Type
age, hours-per-week	continuous
workclass, education, gender, occupation	categorical

A **binary classification** task — the categorical features can't be fed to a model as-is.

Why Not Just Use Integers?

A linear model computes $\hat{y} = w[0] \cdot x[0] + \dots + w[p] \cdot x[p] + b$.

- Encoding `education` as 0, 1, 2, 3... implies a **false ordering & spacing**
- The model would treat "Masters = 3" as *three times* "Bachelors = 1"
- There is **no meaningful order** between categories

We need a representation where the numbers make sense in that formula.

One-Hot Encoding (Dummy Variables)

Replace one categorical feature with **one 0/1 column per category**.

workclass	Gov	Private	Self-emp	Self-emp-inc
Private	0	1	0	0
Self-emp	0	0	1	0

- Exactly **one** column is 1 per row ("one-out-of-N")
- Drop the original column, keep the 0/1 features

One-Hot Encoding with pandas

`get_dummies` encodes every **object/string** (or categorical) column automatically.

```
import pandas as pd
data.gender.value_counts()          # inspect a column's values first
data_dummies = pd.get_dummies(data) # continuous cols untouched
```

- On `adult`: 6 columns → **46** columns after encoding
- A logistic regression on the result scores **0.81** on the test set

! Encode Train & Test Together

Categories must map to the **same columns** in train and test.

- If a category appears in train but not test → **different number of columns**
- Worse: same count but **shifted meaning** — "Government" column now encodes "Self-employed"

Fix: call `get_dummies` on the combined data, **or** align columns afterward

(`test.reindex(columns=train.columns, fill_value=0)`).

! Don't Leak the Target

`get_dummies` also encodes the **target** if it's a string column.

```
# adult: income becomes two columns income_<=50K, income_>50K
features = data_dummies.loc[:, 'age':'occupation_Transport-moving']
X = features.values
y = data_dummies['income_>50K'].values
```

Including the output (or anything derived from it) in `X` is a **common, silent mistake**.

Numbers Can Encode Categoricals

A column of integers may really be **categorical** (e.g. survey codes 0–8 for `workclass`).

- `get_dummies` treats **numbers as continuous** — it won't encode them
- The right encoding depends on whether an **order** exists:
 - no order (workclass) → **must** one-hot encode
 - ordered (star ratings) → depends on task & model

Forcing One-Hot on Integer Columns

Two ways to encode integer-coded categories:

```
# Option A - cast to string, then get_dummies
demo_df['Integer Feature'] = demo_df['Integer Feature'].astype(str)
pd.get_dummies(demo_df, columns=['Integer Feature', 'Categorical Feature'])

# Option B - scikit-learn's OneHotEncoder
from sklearn.preprocessing import OneHotEncoder
```

Or pass `columns=[...]` to `get_dummies` to name exactly which to encode.

Part 2

Transforming Features: Binning, Interactions, Polynomials

Binning (Discretization)

The best representation depends on the **model**. On the 1-feature `wave` data:

- A **linear model** learns only a straight line
- A **decision tree** learns a flexible step function

Binning splits a continuous feature into intervals → a categorical feature, letting a **linear model** capture nonlinearity.

Binning in Code

```
import numpy as np
bins = np.linspace(-3, 3, 11)          # 11 edges → 10 bins
which_bin = np.digitize(X, bins=bins)  # which bin each point falls in

from sklearn.preprocessing import OneHotEncoder
X_binned = OneHotEncoder(sparse=False).fit_transform(which_bin)
X_binned.shape                          # (100, 10)
```

Each point is now represented by **which of 10 bins** it lands in.

The Effect of Binning

After binning, a linear model and a decision tree make the **same** prediction — a constant within each bin.

- **Linear model:** gained flexibility → a different value per bin ✓
- **Decision tree:** lost flexibility — it could already split anywhere ✗

*Binning helps **linear models** on large, high-dim data with nonlinear effects.
It gives tree-based models little.*

Interactions: Adding a Slope

Binning gives each bin a flat value. Add the **original feature** back so the line can slope:

```
X_combined = np.hstack([X, X_binned])    # (100, 11)
```

- Now the model learns an **offset per bin + one shared slope**
- But the slope is the **same** across all bins — often not enough

Interactions: A Slope *per* Bin

Add a **product** (interaction) feature = original feature $\times$ bin indicator:

```
X_product = np.hstack([X_binned, X * X_binned])    # (100, 20)
```

- Each bin gets its **own offset** *and* **its own slope**
- "Interaction features" let the effect of one feature **depend on** another

Polynomial Features

Add powers of a feature: $x, x^2, x^3, \dots, x^{10}$.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=10, include_bias=False)
X_poly = poly.fit_transform(X)      # 1 feature → 10 features
poly.get_feature_names()           # ['x0', 'x0^2', ..., 'x0^10']
```

Combined with a linear model → classic **polynomial regression**.

Polynomial Regression

- Yields a **smooth** fit on the 1-D `wave` data
-  High-degree polynomials swing to **extremes** at the boundaries / sparse regions
- A **kernel SVM** (`SVR`) reaches a similar smooth fit with **no explicit** feature expansion

```
from sklearn.svm import SVR
SVR(gamma=1).fit(X, y)
```

Polynomials on Real Data (Boston)

Scale, then add all degree-2 interactions: **13** → **105 features**.

```
X_scaled = MinMaxScaler().fit_transform(X_train)
X_poly = PolynomialFeatures(degree=2).fit_transform(X_scaled)
```

- **Ridge:** 0.621 → **0.753** — interactions help a lot
- **Random forest:** 0.799 → 0.763 — interactions *hurt* slightly

Complex models find interactions themselves; linear models need them added.

Univariate Nonlinear Transformations

Functions like **log**, **exp**, **sin** reshape a single feature's distribution.

- Help **linear models & neural nets**; irrelevant to **tree-based** models
- Goal: make features (and regression targets) look roughly **Gaussian**
- `sin` / `cos` suit **periodic** data

Most models work best when each feature has a bell-curve-ish shape.

Log Transform for Count Data

Count data (many small values, few huge ones — Poisson-like) is hard for linear models.

```
X_train_log = np.log(X_train + 1)      # +1 because log(0) is undefined
```

- Ridge on raw counts: $R^2 = 0.622$
- Ridge on **log-transformed** counts: $R^2 = 0.875$

Transforming the **target** y with $\log(y + 1)$ often helps too.

Part 3

Automatic Feature Selection

Why Select Features?

More features → more complex model → higher chance of **overfitting**.

- Removing uninformative features → simpler models that **generalize** better
- Three strategies: **univariate**, **model-based**, **iterative**
- All are **supervised** (need y) → fit selection on the **training set only**

Demo setup: cancer (30 features) + 50 noise features = 80, then select.

1 • Univariate Statistics

Test each feature's relationship to the target **independently** (ANOVA, `f_classif`).

```
from sklearn.feature_selection import SelectPercentile
sel = SelectPercentile(percentile=50).fit(X_train, y_train) # 80 → 40
```

- Very **fast**, model-independent
- ✗ Misses features useful only **in combination**
- Logistic regression: 0.930 (all) → **0.940** (selected)

2 · Model-Based Selection

Use a model's importance scores to keep the best features — considers all **together**.

```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
sel = SelectFromModel(
    RandomForestClassifier(n_estimators=100),
    threshold='median').fit(X_train, y_train)    # keeps ~half
```

- Uses `feature_importances_` (trees) or `coef_` (linear/L1)
- Logistic regression on the selection: **0.951**

3 · Iterative Selection (RFE)

Recursive Feature Elimination: fit, drop the weakest feature, repeat.

```
from sklearn.feature_selection import RFE
sel = RFE(RandomForestClassifier(n_estimators=100),
          n_features_to_select=40).fit(X_train, y_train)
```

- Most **expensive** (trains the model many times)
- Best selection here → logistic regression: **0.951**
- The RF *inside* RFE also scores 0.951 — good features close the gap

Comparing the Three Methods

Method	Considers	Cost	scikit-learn
Univariate	one feature at a time	cheap	<code>SelectPercentile</code>
Model-based	all features together	medium	<code>SelectFromModel</code>
Iterative (RFE)	all, step by step	expensive	<code>RFE</code>

Real-world gains are usually modest — but selection speeds up prediction and aids interpretability.

Part 4

Utilizing Expert Knowledge

Utilizing Expert Knowledge

Domain knowledge suggests features the raw data can't reveal.

- **Flight prices:** the date alone won't tell a model about **holidays**
- Add a "holiday / school-break" flag → prior knowledge the model can use
- Adding a feature never forces the model to use it — if useless, **no harm**

The best features often come from **understanding the problem**.

Case Study: Citi Bike Rentals

Predict bike rentals per 3-hour interval. First try: a single **POSIX-time** feature + random forest.

```
X = citibike.index.astype("int64").reshape(-1, 1)    # seconds since 1970
RandomForestRegressor().fit(...)                    # Test R2 = -0.04 🤖
```

- Test timestamps are **outside** the training range
- **Trees can't extrapolate** → they predict the last seen value → useless

Adding the Right Features

Swap POSIX time for features that actually capture the pattern:

Features (random forest)	Test R ²
POSIX time	−0.04
hour of day	0.60
hour + day of week	0.84

Two "common sense" features transform the model — and require **no** extra complexity.

Encoding Matters for Linear Models

Same features, a `LinearRegression` — the encoding decides everything:

Encoding of hour & day	Test R ²
integers (treated as continuous)	0.13
one-hot	0.62
one-hot + interactions	0.85

Integers imply "later = linearly more"; one-hot + interactions matches the random forest — and stays **interpretable** (one coefficient per day×hour).

Chapter Summary

- **One-hot encode** categoricals; encode train & test alike; never leak the target
- Watch for **integer-coded** categories — cast to string / use `OneHotEncoder`
- **Binning, interactions, polynomials** give linear models nonlinear power
- **Log/other transforms** tame skewed & count features
- **Select features** (univariate → model-based → iterative) to simplify
- **Expert knowledge** and the right **encoding** often matter most

Next: Chapter 5 — Model Evaluation & Improvement